Análise de Eficácia de Testes com Teste de Mutação

Engenharia de Software

Victor Ferreira de Almeida

Projeto: operacoes-mutante-lab — biblioteca de 50 operações aritméticas em src/operacoes.js
Ferramentas: Jest 29 e StrykerJS (@stryker-mutator/core, @stryker-mutator/jest-runner)

1. Análise Inicial

1.1 Cobertura de código (*baseline*)

Comando executado:

```
npm test -- --coverage
```

Métrica	Valor
Statements	85,41%
Branches	58,82%
Functions	100%
Lines	98,64%

O único arquivo de produção do projeto, src/operacoes.js, foi o objeto da análise.

A suíte inicial, em test/operacoes.test.js, continha **50 testes** (um por função), todos aprovados. A linha 112 (mediana com tamanho par) permanecia parcialmente descoberta quanto aos ramos, e diversos caminhos de erro (throw) não eram exercitados.

1.2 Pontuação de mutação inicial

O arquivo stryker.conf.json foi configurado com o *runner* Jest, mutação em src/**/*.js e relatório HTML em reports/mutation/mutation.html.

Comando executado:

```
npx stryker run
```

Indicador	Valor
Mutation Score	**73,71%**
Mutantes gerados	213
Mortos (*killed*)	154
Timeout	3
Sobreviventes	44
Sem cobertura	12

1.3 Cobertura × eficácia

Com **98,64% de linhas** e **100% de funções** cobertas, a suíte parecia completa segundo a cobertura tradicional. No entanto, o *mutation score* de **73,71%** indica que cerca de **um quarto dos mutantes** permaneceu vivo: os testes executavam o código, mas **não detectavam alterações semânticas** (operadores trocados, condições simplificadas, mensagens de erro removidas, etc.).

Esse resultado ilustra o argumento central do trabalho: **uma cobertura próxima de 100% não garante testes eficazes**. A cobertura mede a *execução* do código; o teste de mutação mede a *capacidade da suíte de falhar* quando o programa está

incorreto.

---

2. Análise de Mutantes Críticos (primeira execução)

Entre os 44 mutantes sobreviventes iniciais, foram selecionados três casos representativos.

2.1 ArithmeticOperator — celsiusParaFahrenheit

Código original:

```
function celsiusParaFahrenheit(celsius) { return (celsius * 9/5) + 32; }
```

Mutações (exemplos): (celsius * 9 * 5) + 32 e (celsius / 9/5) + 32.

Por que o teste original não detectou o mutante: o teste 38 verificava apenas celsiusParaFahrenheit(0) === 32. Tanto a implementação correta quanto várias fórmulas incorretas produzem 32 para a entrada 0; por isso, o mutante sobrevive.

2.2 ConditionalExpression — isPar

Código original:

```
function isPar(n) { return n % 2 === 0; }
```

Mutação: return true; (condição substituída por uma constante verdadeira).

Por que o teste original não detectou o mutante: o teste 15 apenas afirmava isPar(100) === true, caso em que um mutante que sempre retorna true também passa. Faltavam um caso falso (número ímpar) e o caso 0.

2.3 StringLiteral — divisao

Código original:

```
if (b === 0) throw new Error('Divisão por zero não é permitida.');
```

Mutação: throw new Error("").

Por que o teste original não detectou o mutante: o teste 4 usava expect(() => divisao(5, 0)).toThrow() sem validar a mensagem. Qualquer Error, inclusive com texto vazio, satisfaz essa asserção.

---

3. Solução Implementada

Foi criada a suíte tests/operacoes.mutacao.test.js, com **19 testes reforçados**, mantendo-se a suíte original (mais fraca) para fins de comparação histórica. Os exemplos a seguir estão diretamente associados aos mutantes analisados na Seção 2.

3.1 Conversão térmica (elimina ArithmeticOperator)

```
test('conversões térmicas com valores não triviais', () => {
  expect(celsiusParaFahrenheit(100)).toBeCloseTo(212);
  expect(celsiusParaFahrenheit(-40)).toBeCloseTo(-40);
  expect(fahrenheitParaCelsius(212)).toBeCloseTo(100);
  expect(fahrenheitParaCelsius(-40)).toBeCloseTo(-40);
});
```

Justificativa: a conversão de 100 °C para 212 °F e o ponto de equivalência entre as escalas (−40 °C = −40 °F) invalidam mutações que alteram a ordem ou o tipo dos operadores * e /.

3.2 Paridade (elimina ConditionalExpression em isPar / isImpar)

```
test('paridade distingue par, ímpar e zero', () => {
  expect(isPar(4)).toBe(true);
  expect(isPar(3)).toBe(false);
  expect(isPar(0)).toBe(true);
  expect(isImpar(3)).toBe(true);
  expect(isImpar(4)).toBe(false);
  expect(isImpar(0)).toBe(false);
});
```

Justificativa: a verificação de resultados false impede a sobrevivência de mutantes que retornam sempre true.

3.3 Mensagem de erro na divisão (elimina StringLiteral)

```
test('divisão por zero deve lançar mensagem específica', () => {
  expect(() => divisao(1, 0)).toThrow('Divisão por zero não é permitida.');
});
```

Justificativa: a asserção vincula o comportamento ao texto esperado, e não apenas à existência de uma exceção.

3.4 Outras melhorias relevantes

- **Observabilidade:** jest.spyOn(Array.prototype, 'reduce') garante que produtoArray([]) utilize o retorno antecipado, e não apenas o acumulador padrão de reduce.
- **Fronteiras:** clamp com intervalos invertidos (min > max) e Object.is(clamp(-0, -1, 0), -0) permitem distinguir > de >= no teto superior.
- **Mediana:** arrays desordenados e de tamanho par ([4, 1, 2, 3] → 2,5) cobrem a ordenação e o ramo par.
- **Comparações:** casos de empate e de desigualdade em isMaiorQue, isMenorQue e isEqual.

---

4. Resultados Finais

4.1 Cobertura de código (final)

Métrica	Valor
Statements	100%
Branches	100%
Functions	100%
Lines	100%

4.2 Pontuação de mutação (final)

Indicador	Valor
Mutation Score	**98,12%**
Mortos (*killed*)	206
Timeout	3
Sobreviventes	4
Sem cobertura	0

Relatório HTML: reports/mutation/mutation.html.

4.3 Mutantes remanescentes (equivalentes)

Restam quatro mutantes na linha 19 da função fatorial:

```
if (n === 0 || n === 1) return 1;
```

As mutações if (false) return 1, if (n === 0 && n === 1) return 1, if (false || n === 1) return 1 e if (n === 0 || false) return 1 produzem o mesmo resultado observável que o código original para todos os valores válidos de entrada, pois o laço subsequente também retorna 1 para n = 0 e n = 1. Trata-se, portanto, de **mutantes equivalentes**, que não podem ser eliminados por testes de caixa-preta sem alterar o código de produção.

4.4 Evolução resumida

Fase	Mutation Score	Testes
Baseline	73,71%	50
Intermediária	96,71%	64
Final	98,12%	69

---

5. Conclusão

O teste de mutação complementa a cobertura de código ao simular defeitos realistas (mutantes) e verificar se a suíte reage a eles. Neste laboratório, a cobertura inicial já era elevada, mas o *mutation score* era baixo — evidência de asserções fracas, ausência de casos de borda e validação superficial de erros.

Ao reforçar as asserções (mensagens de erro, resultados falsos, fórmulas com múltiplos pontos de verificação e, quando necessário, *spies* de interação), o *mutation score* passou de 73,71% para 98,12%, superando a meta de 98%. Os mutantes remanescentes em fatorial evidenciam outra lição: nem todo mutante sobrevivente indica um teste deficiente; parte deles pode ser equivalente ao comportamento correto.

Em projetos reais, combinar cobertura de código, teste de mutação e análise de mutantes equivalentes oferece uma visão muito mais confiável da eficácia dos testes do que qualquer métrica isolada.